

LESSON 2 Data, Regression, and Machine Learning; Python

In Lesson 1, we looked at the rows and columns of a matrix and discussed matrix product and factorization, reaching CR factorization and the four fundamental subspaces of a matrix. In this lesson, we will bring in **data** and see how matrix methods are important in **data analysis** and **machine learning**.

2.1 Regression and Machine Learning

Suppose we have the following data set on 3 houses in the same neighborhood.

Living Area (1000 sq ft) x	1	2.25	1.5
Price (100,000) y	4	6	5

Here is the **regression** problem in data analysis: based on a set of paired data $(x^{(i)}, y^{(i)})$, find how y (price) relates to x (living area). Regression is also a basic topic in machine learning. Just what exactly is machine learning? Of the gazillions of definitions, the following seems to be as good as any (IBM):

Machine learning is a branch of computer science which focuses on the use of *data* and *algorithms* to imitate the way that humans learn, gradually improving its accuracy.

In our example, we may learn from (existing) data to construct a **predictor** function h which predicts the price y (output or **target** variable) of a new house given living area x , (input or **feature** variable). You may be used to seeing regression with a different vocabulary, but we have adopted here the ML vocabulary.

2.2 Review of Least Square Solution and Normal Equation

In this example, there is only one feature variable and the simplest function h is linear: $h(x) = \theta x$, where we may find the **parameter** θ by fitting our data to the model, i.e. plugging in $h(x^{(i)}) = \theta x^{(i)} = y^{(i)}$ for all 3 pairs $x^{(i)}, y^{(i)}$ in the data set. We get a linear system or a matrix equation, $X\theta = Y$, which turns out to be inconsistent:

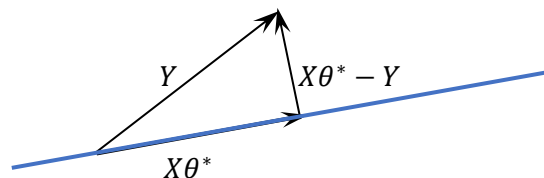
$$\begin{array}{l} \theta = 4 \\ 2.25\theta = 6 \\ 1.5\theta = 5 \end{array} \Leftrightarrow \begin{array}{c} \left[\begin{array}{c|c} 1 & 4 \\ 2.25 & 6 \\ 1.5 & 5 \end{array} \right] \theta = \left[\begin{array}{c} 4 \\ 6 \\ 5 \end{array} \right], \\ X \quad Y \end{array} \quad \left[\begin{array}{c|c} 1 & 4 \\ 2.25 & 6 \\ 1.5 & 5 \end{array} \right] \rightarrow \left[\begin{array}{c|c} 1 & 4 \\ 0 & -3 \\ 0 & -1 \end{array} \right] \quad 0 = -3$$

The column picture says that Y is not in the **column space** of X (blue line), there is no θ such that $X\theta = Y$. Gaussian elimination can go no further, but we learned in MATH2331 that we may find the **best** θ , that is, θ^* such that $\|X\theta^* - Y\| \leq \|X\theta - Y\|$ for all θ . From the picture, $X\theta^*$ is the **projection** of Y onto $V = C(X)$, and $Y - X\theta^*$ is *perpendicular* to $C(X)$ or in the nullspace $N(X^T)$, resulting in the **normal equation** of $X\theta = Y$:

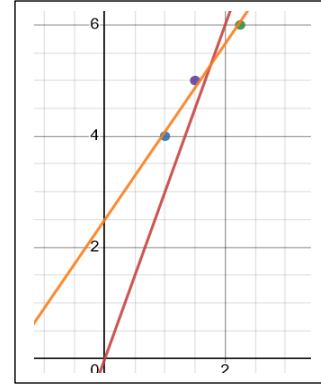
$$\begin{aligned} X^T(X\theta^* - Y) &= 0 \\ \Rightarrow X^T X\theta^* - X^T Y &= 0 \\ \Rightarrow \mathbf{X^T X\theta^* = X^T Y} \end{aligned}$$

For our example,

$$X^T X = [1 \ 2.25 \ 1.5] \begin{bmatrix} 1 \\ 2.25 \\ 1.5 \end{bmatrix} = 8.3125, \quad X^T Y = [1 \ 2.25 \ 1.5] \begin{bmatrix} 4 \\ 6 \\ 5 \end{bmatrix} = 25$$



The normal equation is: $8.3125\theta^* = 25$, and $\theta^* = 3$. The **red line** $h(x) = 3x$ is the best line through the origin which fits the data. There is a line (**orange**) closer to the data points than the red line if we allow nonzero intercept. In fact, when we say linear model, we mean $h(x) = \theta_0 + \theta_1 x$, which is an **affine** function (θ_0 can be nonzero).



The transition from one parameter θ to two parameters θ_0, θ_1 does not change the form of the *matrix equation* $X\theta = Y$, but note that now $\theta = (\theta_0, \theta_1)$ is a vector, not a number.

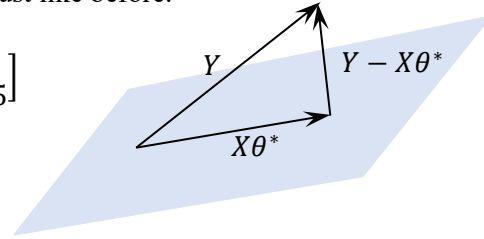
$$\begin{array}{rcl} \theta_0 + \theta_1 = 4 \\ \theta_0 + 2.25\theta_1 = 6 \\ \theta_0 + 1.5\theta_1 = 5 \end{array} \Leftrightarrow \begin{array}{cc|c} 1 & 1 & 4 \\ 1 & 2.25 & 6 \\ 1 & 1.5 & 5 \end{array} \begin{array}{c} [\theta_0] \\ [\theta_1] \end{array} = \begin{array}{c} [4] \\ [6] \\ [5] \end{array}, \quad \begin{array}{cc|c} 1 & 1 & 4 \\ 1 & 2.25 & 6 \\ 1 & 1.5 & 5 \end{array} \rightarrow \begin{array}{cc|c} 1 & 1 & 4 \\ 0 & 1 & 2 \\ 0 & 0 & -0.5 \end{array}$$

$X \quad \theta \quad Y$

There is no solution θ for the system: $(4, 6, 5)$ is *not* in $V = C(X)$, which is the 2D blue plane spanned the two independent columns of X . Again, we find a vector θ^* such that $X\theta^*$, which is in $C(X)$, is as close as possible to Y , and again $X\theta^* = \text{proj}_V Y$ and $Y - X\theta^*$ is perpendicular to $C(X)$, or $Y - X\theta^*$ is in the nullspace of X^T , that is $X^T(X\theta^* - Y) = 0$, and we get the normal equation $X^T X \theta^* = X^T Y$ just like before.

$$X^T X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2.25 & 1.5 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2.25 \\ 1 & 1.5 \end{bmatrix} = \begin{bmatrix} 3 & 4.75 \\ 4.75 & 8.3125 \end{bmatrix}$$

$$X^T Y = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2.25 & 1.5 \end{bmatrix} \begin{bmatrix} 4 \\ 6 \\ 5 \end{bmatrix} = \begin{bmatrix} 15 \\ 25 \end{bmatrix}$$



The normal equation $\begin{bmatrix} 3 & 4.75 \\ 4.75 & 8.3125 \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} = \begin{bmatrix} 15 \\ 25 \end{bmatrix}$ has solution $\theta^* = \begin{bmatrix} 2.5 \\ 1.58 \end{bmatrix}$, which gives the orange line: $h(x) = 2.5 + 1.58x$.

2.3 Multi-Variable Linear Regression

We may include more input or **feature** variables such as number of bedrooms, lot size, etc. Suppose we have d features and n data points:

Living Area	Bedrooms	...		Price
$x_1^{(1)}$	$x_2^{(1)}$	...	$x_d^{(1)}$	$y^{(1)}$
$\vdots$	$\vdots$		$\vdots$	$\vdots$
$x_1^{(n)}$	$x_2^{(n)}$	...	$x_d^{(n)}$	$y^{(n)}$

We want to construct a linear model for y :

$$h(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_d x_d$$

This leads to the following inconsistent linear system:

$$\begin{array}{rcl} \theta_0 + \theta_1 x_1^{(1)} + \dots + \theta_d x_d^{(1)} = y^{(1)} \\ \vdots \\ \theta_0 + \theta_1 x_1^{(n)} + \dots + \theta_d x_d^{(n)} = y^{(n)} \end{array} \Leftrightarrow \begin{array}{ccccc} 1 & x_1^{(1)} & & x_d^{(1)} & \\ \vdots & \vdots & \dots & \vdots & \\ 1 & x_1^{(n)} & & x_d^{(n)} & \end{array} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

$X \quad \theta \quad Y$

Every parameter θ_i corresponds a feature variable x_i , except the constant parameter θ_0 . We find the best parameters θ^* by solving the normal equation of $X\theta = Y$:

$$(X^T X)\theta^* = X^T Y$$

A typical data matrix X have **linearly independent columns**, and there is a unique “best” solution due to the theorem:

- For any matrix A , $A^T A$ is invertible if and only if A has independent columns.

Proof ($\Rightarrow$) If $Ax = 0$, then $A^T(Ax) = 0$ and $(A^T A)x = 0$. Since $A^T A$ is invertible, $x = 0$, so A has independent columns.

($\Leftarrow$) If $(A^T A)x = 0$, then $x^T(A^T A)x = (Ax)^T Ax = \|Ax\|^2 = 0$, which implies that $Ax = 0$. Since A has independent columns, $x = 0$, and $A^T A$ has independent columns, but since $A^T A$ is square, so A is invertible.

Notice how matrices and vectors are the natural “objects” we may use to store data and work with data. Furthermore, various linear algebra concepts such as projection onto a subspace (the column space of a matrix) and orthogonality give us a way to understand the linear regression problem, this is beyond Gaussian elimination, which after concluding that a system is inconsistent can do nothing further!

Linear regression is also called **least squares** regression because we minimize the square (Euclidean) distance $\|Y - X\theta\|^2$:

$$\begin{aligned} Y - X\theta &= \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix} - \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_d^{(1)} \\ \vdots & \vdots & & \vdots \\ 1 & x_1^{(n)} & \cdots & x_d^{(n)} \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix} - \begin{bmatrix} -(x^{(1)})^T \\ \vdots \\ -(x^{(n)})^T \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} \\ &= \begin{bmatrix} y^{(1)} - (x^{(1)})^T \theta \\ \vdots \\ y^{(n)} - (x^{(n)})^T \theta \end{bmatrix} = \begin{bmatrix} y^{(1)} - \theta^T x^{(1)} \\ \vdots \\ y^{(n)} - \theta^T x^{(n)} \end{bmatrix} \quad \begin{matrix} (x^{(1)})^T \theta = \theta^T x^{(1)} \\ \text{because } u^T v = v^T u \end{matrix} \end{aligned}$$

The square distance $\|Y - X\theta\|^2$ or the **loss function** is:

$$\|Y - X\theta\|^2 = (y^{(1)} - \theta^T x^{(1)})^2 + \cdots + (y^{(n)} - \theta^T x^{(n)})^2 = \sum_{i=1}^n (y^{(i)} - \theta^T x^{(i)})^2$$

The mean least square distance, called the mean **residual sum of squares (RSS)**, a measure of the **error** of the model. Recall the first model in Section 2.1, $y = h(x) = \theta x$, with a single parameter θ , where the best parameter is $\theta^* = 3$, the mean RSS is:

$$\frac{1}{n} \|Y - X\theta^*\|^2 = \frac{1}{3} \left\| \begin{bmatrix} 4 \\ 6 \\ 5 \end{bmatrix} - \begin{bmatrix} 1 \\ 2.25 \\ 1.5 \end{bmatrix} 3 \right\|^2 = \frac{1}{3} (1^2 + 0.75^2 + 0.5^2) = 0.604$$

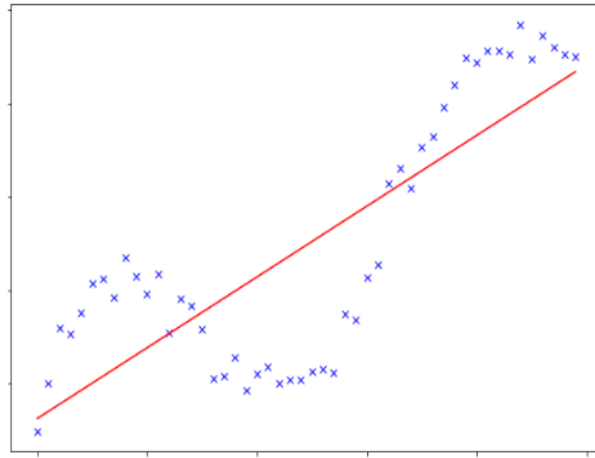
For our second model, $y = h(x) = \theta_0 + \theta_1 x$, there are two parameters, θ_0 and θ_1 , or a parameter vector $\theta = (\theta_0, \theta_1)$. The best parameter is $\theta^* = (2.5, 1.58)$, and the mean RSS is

$$\frac{1}{n} \|Y - X\theta\|^2 = \frac{1}{3} \left\| \begin{bmatrix} 4 \\ 6 \\ 5 \end{bmatrix} - \begin{bmatrix} 1 & 1 \\ 1 & 2.25 \\ 1 & 1.5 \end{bmatrix} \begin{bmatrix} 2.5 \\ 1.58 \end{bmatrix} \right\|^2 = 0.008775$$

As expected, the error is much smaller in the second model, when we have more parameters to fit the data.

2.4 Polynomial Regression; Python

We have described **linear** regression, where we built a linear model for the data. For the data below, it makes sense to fit to a higher degree polynomial.



Interestingly, **polynomial regression** results in a linear system, as illustrated below: fitting a set of 4 data points $(x^{(i)}, y^{(i)})$'s: $(-1,1), (0,0), (1,0), (2,2)$ to a quadratic polynomial,

$$y = \theta_0 + \theta_1 x + \theta_2 x^2$$

yields an inconsistent linear system $X\theta = Y$, as in linear regression, but now the feature variables are $x_1 = x$ and $x_2 = x^2$.

$$\begin{aligned} c_1 + c_2(-1) + c_3(-1)^2 &= 1 \\ c_1 + c_2(0) + c_3(0)^2 &= 0 \\ c_1 + c_2(1) + c_3(1)^2 &= 0 \\ c_1 + c_2(2) + c_3(2)^2 &= 2 \end{aligned} \quad \Rightarrow \quad \begin{array}{ccc} 1 & x & x^2 \\ \begin{bmatrix} 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix} &= & \begin{bmatrix} 1 \\ 0 \\ 0 \\ 2 \end{bmatrix} \end{array}$$

We use **Python** to find the least square solution of $X\theta = Y$ by solving the normal equation $(X^T X)\theta^* = X^T Y$ and use Desmos to plot the quadratic model

$$y = -0.15 - 0.45x + 0.75x^2$$

When there are two independent variable x_1 and x_2 , a quadratic model will include cross terms $x_1 x_2$, and a cubic model will have terms like $x_1^2 x_2$ etc., as we will see in homework.

```
# Importing libraries for matrices, array, linear algebra
import numpy as np

# Least squares using the normal equation (A.T*A)*x = (X.T)*y
def LeastSquares(A,b):
    x=np.linalg.solve(A.T @ A, A.T @ b)
    return x

X = np.array([[1, -1, 1], [1, 0, 0], [1, 1, 1], [1, 2, 4]])
Y = np.array([1, 0, 0, 2])
theta = LeastSquares(X,Y)
print("Least squares solution: theta =", theta)

Least squares solution: theta = [-0.15 -0.45  0.75]
```

